LEAD GENERATION & TASK MANAGER MOBILE APP

Complete REST API Reference, Integration Specifications & JSON Payloads

TARGET CLIENTS: iOS 14+ & Android 10+ [React Native, Flutter, Swift, Kotlin]
SPEC VERSION: 2.0 [Production Verified]
AUTHENTICATION: RFC 7519 JWT Bearer Tokens [HMAC-SHA256]
DOCUMENT STATUS: **Fully Implemented & Tested**
BASE ENDPOINT: `https://flux-india.co.in/api/ or http://<host>:8000/api/`
SECURITY TIER: Enterprise Role-Based Access [RBAC]

EXECUTIVE SUMMARY & CORE MOBILE ARCHITECTURE

1. Multi-User Lead Assignment Architecture:
Leads are linked to sales representatives via the relational mapping table 'lead_assignments'. Multiple representatives can simultaneously manage and collaborate on the same client company without data overwriting.

2. Independent Task & Interaction Histories:
Each assigned user maintains their own independent 'LeadTask' instance and interaction log [Call, WhatsApp, Email]. When one user is removed from a lead, it disappears from only that user's view while other assignees retain full access.

3. Automated 24-Hour Follow-up Reminder Engine:
The system automatically computes reminder_date = next_followup_date - 1 day, delivering timely alerts.

REST API DIRECTORY & SECTION INDEX

SECTION 1: AUTHENTICATION & SESSION MANAGEMENT - Page 2
* POST /api/auth/token/ - Authenticate credentials and obtain JWT access & refresh token pair
* POST /api/auth/token/refresh/ - Renew expired JWT access token without re-login
* GET /api/auth/me/ - Fetch authenticated user profile, companies, and permissions
* POST /api/auth/logout/ - Invalidate session and revoke/blacklist refresh token

SECTION 2: LEAD GENERATION & MANAGEMENT - Pages 3 - 4
* GET /api/leads/ - Paginated leads list with dynamic search, multi-filters, and segment counts
* GET /api/leads/filters/ - Dynamic filter options [cities, segments, categories, assignable users]
* POST /api/leads/ - Create new lead with auto-assignment and duplicate conflict checks
* GET /api/leads/{id}/ - Retrieve detailed lead profile, assigned team members, and audit history
* PATCH /api/leads/{id}/ - Partial update of contact information, notes, and assignees
* DELETE /api/leads/{id}/ - Delete lead record and cascade linked assignments and tasks
* POST /api/leads/{id}/status/ - Quick status transition [Lead, HostLead, Converted, Unassigned]
* POST /api/leads/bulk-status/ - Batch status updates and mass assignment management

SECTION 3: MULTIPLE-USER LEAD ASSIGNMENT & REMOVAL - Page 5
* POST /api/leads/assign/ - Bulk assign 1 or more leads to multiple users simultaneously
* GET /api/leads/{id}/assignments/ - List all representatives currently assigned to a lead
* DELETE /api/leads/{id}/assignments/{uid}/ - Remove specific representative from lead assignment

SECTION 4: TASK MANAGER [CRUD & STATUS] - Pages 6 - 7
* GET /api/tasks/ - View assigned tasks with status counts [All, Pending, Completed]
* POST /api/tasks/ - Create task manually for an assigned lead
* GET /api/tasks/{id}/ - Retrieve task details, client info, schedule dates, and notes
* PATCH /api/tasks/{id}/ - Update task description, assigned date, and next follow-up date
* POST /api/tasks/{id}/status/ - Toggle task status between Pending and Completed
* DELETE /api/tasks/{id}/ - Soft delete task from active task list
* GET /api/tasks/available-leads/ - Fetch leads assigned to user available for task creation

SECTION 5: LEAD COMMUNICATION & FOLLOW-UP - Page 8
* POST /api/tasks/{id}/calls/ - Record interaction [Call / WhatsApp / Email] and next follow-up
* GET /api/tasks/{id}/history/ - Retrieve chronological communication history for this task
* GET /api/leads/{id}/interactions/ - 360-degree aggregated interaction log across all assignees

SECTION 6: REMINDERS & MOBILE PUSH TOKENS - Page 9
* GET /api/tasks/notifications/ - Pending follow-up reminders due today or overdue
* POST /api/notifications/device-token/ - Register mobile FCM/APNS push token
* DELETE /api/notifications/device-token/ - Unregister mobile device push token on sign-out

1. AUTHENTICATION & SESSION MANAGEMENT APIS [JWT & RBAC]

POST /api/auth/token/ -- User Login & Token Generation [Role: Public]

Purpose: Authenticates user credentials and returns JWT access and refresh token pair along with full user profile.
Headers: Content-Type: application/json

REQUEST BODY [JSON]:

```
{
  "username": "dhruvi_patel",
  "password": "UserPass@123"
}
```

SUCCESS RESPONSE [200 OK]:

```
{ "access": "eyJhbGciOi...", "refresh": "eyJ...",
 "token_type": "Bearer", "expires_in": 86400,
 "user": { "id": 10, "username": "dhruvi_patel",
  "full_name": "Dhruvi Patel", "role": "User" } }
```

Error Responses: 401 Unauthorized: {"error": "Incorrect password."} | 400 Bad Request: {"error": "Missing credentials"}

Notes: Store access token in secure mobile storage. Pass 'Authorization: Bearer <access_token>' for all protected endpoints.

POST /api/auth/token/refresh/ -- Renew Expired Access Token [Role: Public / Refresh]

Purpose: Issues a new JWT access token using a valid, non-blacklisted refresh token without requiring re-authentication.
Headers: Content-Type: application/json

REQUEST BODY [JSON]:

```
{
  "refresh": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVC..."
}
```

SUCCESS RESPONSE [200 OK]:

```
{
 "access": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVC...",
 "token_type": "Bearer", "expires_in": 86400
}
```

Error Responses: 401 Unauthorized: {"error": "Invalid or expired refresh token."} | 400 Bad Request: missing token

GET /api/auth/me/ -- User Profile & RBAC Permissions [Role: Authenticated]

Purpose: Retrieves authenticated profile, active role, company memberships, and authorized capability flags.
Headers: Authorization: Bearer <access_token>

SUCCESS RESPONSE [200 OK]:

```
{ "id": 10, "username": "dhruvi_patel", "first_name": "Dhruvi", "last_name": "Patel", "full_name": "Dhruvi Patel",
 "email": "dhruvi@sky9.ai", "role": "User", "is_staff": false, "is_superuser": false,
 "companies": [ { "id": 1, "name": "Impression Pharma", "location": "Ahmedabad" } ], "company_ids": [1],
 "permissions": [ "task.manage", "lead.view_assigned", "communication.record" ] }
```

Error Responses: 401 Unauthorized: {"error": "Authentication credentials were not provided or token expired."}

POST /api/auth/logout/ -- User Logout & Token Revocation [Role: Authenticated]

Purpose: Signs out user, invalidates active sessions, and blacklists refresh token in the backend database.
Headers: Authorization: Bearer <access_token> | Content-Type: application/json

OPTIONAL REQUEST BODY [JSON]:

```
{ "refresh": "eyJhbGciOiJIUzI1NiIsInR5c..." }
```

SUCCESS RESPONSE [200 OK]:

```
{ "message": "Logged out successfully." }
```

Security Policy: Token JTI is recorded in 'blacklisted_refresh_tokens'. Further refresh attempts will return HTTP 401.

2. LEAD GENERATION & MANAGEMENT APIS -- PART 1

GET /api/leads/ -- List Leads with Search, Filters & Counts [Admin: All | User: Assigned]

Purpose: Returns paginated leads with real-time multi-filtering [status, city, segment, assignee] & status counts.
Headers: Authorization: Bearer <access_token> | Query: ?status=Lead&city=Ahmedabad&search=Apex&page=1

SUCCESS RESPONSE [200 OK]:

```
{ "total_leads": 4, "current_page": 1, "total_pages": 1, "counts": { "all": 4, "lead": 2, "hostlead": 1, "converted": 1 },
 "results": [ { "id": 1, "company_name": "Apex Healthcare Ltd", "contact_person": "Raj Shah", "phone": "9876543210",
  "city": "Ahmedabad", "status": "Lead", "assigned_to_name": "Dhruvi Patel, Juhi Vanjara",
  "assigned_user_ids": [10, 11], "created_at": "2026-11-04T10:30:00Z" } ] }
```

RBAC Enforcement: Regular users [e.g. Dhruvi] only receive leads where their user ID is in 'assigned_user_ids'.

GET /api/leads/filters/ -- Filter Options & Segment Breakdown [Role: Authenticated]

Purpose: Returns active cities, segments with dynamic lead counts, and assignable users for filter dropdowns.
Headers: Authorization: Bearer <access_token>

SUCCESS RESPONSE [200 OK]:

```
{ "cities": ["Ahmedabad", "Bangalore", "Mumbai"], "statuses": ["Lead", "HostLead", "Converted"],
 "segments": [ {"name": "Pharma", "count": 12}, {"name": "Hospital", "count": 5} ],
 "assignable_users": [ {"id": 10, "username": "dhruvi", "name": "Dhruvi Patel", "role": "User", "count": 4} ] }
```

Mobile UI Usage: Powers top horizontal chip filters and dropdown selectors on the Lead Generation screen.

POST /api/leads/ -- Create New Lead with Auto-Assignment [Role: Admin / Super]

Purpose: Creates lead. Auto-preselects current user if unassigned. Checks duplicate phone/email and creates initial task.
Headers: Authorization: Bearer <access_token> | Content-Type: application/json

REQUEST BODY [JSON]:

```
{
  "company_name": "Apex Healthcare Ltd",
  "first_name": "Raj", "last_name": "Shah",
  "phone": "9876543210", "city": "Ahmedabad",
  "assigned_to": [10, 11], "notes": "New lead"
}
```

SUCCESS RESPONSE [201 CREATED]:

```
{ "message": "Lead created successfully.",
 "lead": { "id": 1, "company_name": "Apex...",
  "assigned_user_ids": [10, 11],
  "status": "Lead", "created_at": "2026..." } }
```

Error Responses: 400 Bad Request: {"error": "Company name or contact person name is required."} | 409 Conflict: Duplicate phone

GET /api/leads/{id}/ -- Complete Lead Profile & Assignees [Assigned User / Admin]

Purpose: Retrieves complete lead record with full assignment history, status audit trail, and assigned representatives.
Headers: Authorization: Bearer <access_token>

SUCCESS RESPONSE [200 OK]:

```
{ "lead": { "id": 1, "company_name": "Apex Healthcare Ltd", "first_name": "Raj", "last_name": "Shah",
 "email": "raj@apexhealth.com", "phone": "9876543210", "city": "Ahmedabad", "status": "Lead",
 "assigned_users": [ {"id": 10, "name": "Dhruvi Patel"}, {"id": 11, "name": "Juhi Vanjara"} ],
 "assignment_history": [ {"assigned_to_name": "Dhruvi Patel", "assigned_at": "2026-11-04T10:30:00Z"} ] } }
```

Error Responses: 404 Not Found: {"error": "Lead not found or access denied."} | 403 Forbidden: Insufficient permissions

2. LEAD GENERATION & MANAGEMENT APIS -- PART 2

PATCH /api/leads/{id}/ -- Update Lead Information & Assignees [Admin: Full | User: Notes]

Purpose: Updates lead fields. Allows adding/removing assigned users via 'assigned_to'. Automatically adjusts tasks.
Headers: Authorization: Bearer <access_token> | Content-Type: application/json

REQUEST BODY [JSON]:

```
{
  "assigned_to": [10, 11, 1],
  "notes": "Added Tech Admin as co-assignee",
  "segment": "Hospital & Diagnostics"
}
```

SUCCESS RESPONSE [200 OK]:

```
{ "message": "Lead updated successfully.",
 "lead": { "id": 1, "company_name": "Apex...",
   "assigned_user_ids": [10, 11, 1],
   "notes": "Added Tech Admin as co-assignee" } }
```

Error Responses: 404 Not Found: {"error": "Lead not found."} | 403 Forbidden: {"error": "Unauthorized modification."}

DELETE /api/leads/{id}/ -- Permanently Delete Lead Record [Role: Admin / Super]

Purpose: Deletes lead record and cascades related lead assignments and tasks cleanly from the database.
Headers: Authorization: Bearer <access_token>

SUCCESS RESPONSE [200 OK]:

```
{ "message": "Lead deleted successfully." }
```

Error Responses: 403 Forbidden: {"error": "Only Admins and Superadmins can delete leads."} | 404 Not Found

POST /api/leads/{id}/status/ -- Quick Status Transition [Assigned User / Admin]

Purpose: Updates status: Lead, HostLead, Converted, or Unassigned. Unassigning safely removes assignments.
Headers: Authorization: Bearer <access_token> | Content-Type: application/json

REQUEST BODY [JSON]:

```
{
  "status": "Converted",
  "status_notes": "Contract finalized."
}
```

SUCCESS RESPONSE [200 OK]:

```
{ "message": "Lead status updated to Converted.",
 "lead": { "id": 1, "status": "Converted",
   "status_updated_at": "2026-11-05T12:00:00Z" } }
```

Special Behavior: If 'Unassigned' is passed, deletes 'lead_assignments' and marks tasks as deleted [Admin only].
Error Responses: 400 Bad Request: {"error": "Invalid status value."} | 403 Forbidden: {"error": "Permission denied."}

POST /api/leads/bulk-status/ -- Batch Status & Mass Assignment Update [Role: Admin / Super]

Purpose: Batch update status or unassigns multiple selected leads simultaneously with audit notes.
Headers: Authorization: Bearer <access_token> | Content-Type: application/json

REQUEST BODY [JSON]:

```
{
  "lead_ids": [1, 2, 3],
  "status": "HostLead",
  "notes": "Bulk moved to HostLead campaign"
}
```

SUCCESS RESPONSE [200 OK]:

```
{ "message": "Successfully updated status for 3 leads",
 "updated_count": 3, "failed_count": 0
}
```

Error Responses: 400 Bad Request: {"error": "lead_ids must be a non-empty list."} | 403 Forbidden: Permission denied

3. MULTIPLE-USER LEAD ASSIGNMENT & REMOVAL APIS

ARCHITECTURE: RELATIONAL MANY-TO-MANY [M2M] ASSIGNMENT MODEL

* All assignments are stored in the relational database table 'lead_assignments' [lead_id, user_id, assigned_by, assigned_at].
* No comma-separated username strings are used. Each assigned representative receives an independent 'LeadTask' instance.
* When user A logs notes or sets a follow-up, it is completely isolated and NEVER overwrites user B's records.

POST /api/leads/assign/ -- Multiple-User Lead Assignment [Role: Admin / Super]

Purpose: Assigns 1 or more leads to multiple users simultaneously. Automatically provisions independent tasks per user.
Headers: Authorization: Bearer <access_token> | Content-Type: application/json

REQUEST BODY [JSON]:

```
{
  "lead_ids": [1],
  "assigned_to": [10, 11, 1],
  "notes": "Co-assignment: Dhruvi, Juhi, Admin"
}
```

SUCCESS RESPONSE [200 OK]:

```
{ "message": "Successfully updated assignment
 "for 1 leads to Dhruvi Patel, Juhi Vanjara,",
 "Tech Admin.",
 "updated_count": 1 }
```

Audit Trail: Creates 'lead_assignment_history' records capturing previous assignee and assignment actor timestamps.
Error Responses: 400 Bad Request: {"error": "No valid leads selected."} | 403 Forbidden: {"error": "Permission denied."}

GET /api/leads/{id}/assignments/ -- List Active Assigned Representatives [Assigned User / Admin]

Purpose: Returns full list of all active users assigned to this lead, with assigned_by and assigned_at timestamps.
Headers: Authorization: Bearer <access_token>

SUCCESS RESPONSE [200 OK]:

```
{ "lead_id": 1, "company_name": "Apex Healthcare Ltd", "total_assigned": 2,
 "assignments": [
  { "user_id": 10, "username": "dhruvi", "name": "Dhruvi Patel", "assigned_by_name": "Tech Admin", "assigned_at": "2026-11-04T10:30:00Z" },
  { "user_id": 11, "username": "juhi", "name": "Juhi Vanjara", "assigned_by_name": "Tech Admin", "assigned_at": "2026-11-04T10:30:00Z" } ] }
```

Error Responses: 404 Not Found: {"error": "Lead not found or access denied."}

DELETE /api/leads/{id}/assignments/{uid}/ -- Remove User Assignment [Role: Admin / Super]

Purpose: Removes specific representative from lead. Lead disappears from only that user's Task Manager.
Headers: Authorization: Bearer <access_token>

SUCCESS RESPONSE [200 OK]:

```
{
  "message": "User Juhi Vanjara removed from lead assignment successfully.",
  "lead_id": 1, "removed_user_id": 11, "remaining_assigned_count": 1
}
```

Data Integrity: Dhruvi Patel retains full access. Juhi's personal Task is soft-deleted [is_deleted=True].
Error Responses: 404 Not Found: {"error": "User is not assigned to this lead."} | 403 Forbidden: {"error": "Permission denied."}

4. TASK MANAGER APIS [CRUD & STATUS] -- PART 1

GET /api/tasks/ -- List Tasks with Status Counts & Auto-Sync [User: Own Tasks | Admin: All]

Purpose: Lists active tasks with dynamic status counts [All, Pending, Completed]. Auto-syncs assigned leads.
Headers: Authorization: Bearer <access_token> | Query: ?status=Pending&search=Apex

SUCCESS RESPONSE [200 OK]:

```
{ "counts": { "all": 10, "pending": 7, "completed": 3 },
 "tasks": [ { "id": 101, "lead_id": 1, "company_name": "Apex Healthcare Ltd", "contact_person": "Raj Shah",
  "phone": "9876543210", "city": "Ahmedabad", "description": "Follow up regarding quotation",
  "assigned_date": "2026-11-04", "next_followup_date": "2026-11-10", "reminder_date": "2026-11-09",
  "status": "Pending", "is_due_today": false, "is_overdue": false, "call_count": 2 } ] }
```

Auto-Sync Rule: Ensures every lead assigned to the user automatically has an active task in their Task Manager.

POST /api/tasks/ -- Create Task Manually for Assigned Lead [Assigned User / Admin]

Purpose: Creates task for an assigned lead. Calculates reminder_date automatically [1 day prior to follow-up].
Headers: Authorization: Bearer <access_token> | Content-Type: application/json

REQUEST BODY [JSON]:

```
{
  "lead_id": 1,
  "description": "Product demo meeting scheduled",
  "next_followup_date": "2026-11-10",
  "future_discussion": "Demonstrate scanner"
}
```

SUCCESS RESPONSE [201 CREATED]:

```
{ "message": "Task created successfully.",
 "task": { "id": 102, "lead_id": 1,
  "company_name": "Apex Healthcare Ltd",
  "next_followup_date": "2026-11-10",
  "reminder_date": "2026-11-09", "status": "Pending" } }
```

Error Responses: 400 Bad Request: {"error": "A pending task already exists."} | 403 Forbidden: {"error": "Lead not assigned."}

GET /api/tasks/{id}/ -- Task Details & Follow-up History [Assigned User / Admin]

Purpose: Retrieves full task details, contact details, scheduled reminder dates, and past call/discussion history.
Headers: Authorization: Bearer <access_token>

SUCCESS RESPONSE [200 OK]:

```
{ "task": { "id": 101, "lead_id": 1, "company_name": "Apex Healthcare Ltd", "contact_person": "Raj Shah",
 "phone": "9876543210", "description": "Follow up regarding quotation", "status": "Pending",
 "next_followup_date": "2026-11-10", "reminder_date": "2026-11-09", "future_discussion": "Confirm order quantity",
 "call_history": [ { "id": 201, "interaction_type": "Call", "discussion": "Discussed initial quote", "user_name": "Dhruvi Patel" } ] } }
```

Error Responses: 404 Not Found: {"error": "Task not found or access denied."}

PATCH /api/tasks/{id}/ -- Update Task Schedule & Discussion Notes [Assigned User / Admin]

Purpose: Edits task description, assigned date, next follow-up date, and future discussion agenda notes.
Headers: Authorization: Bearer <access_token> | Content-Type: application/json

REQUEST BODY [JSON]:

```
{
  "description": "Updated agenda notes",
  "next_followup_date": "2026-11-12",
  "future_discussion": "Review sample pack"
}
```

SUCCESS RESPONSE [200 OK]:

```
{ "message": "Task updated successfully.",
 "task": { "id": 101, "next_followup_date": "2026-11-12",
  "reminder_date": "2026-11-11",
  "future_discussion": "Review sample pack" } }
```

Error Responses: 404 Not Found: {"error": "Task not found."} | 400 Bad Request: Invalid date format

4. TASK MANAGER APIS [CRUD & STATUS] -- PART 2

POST /api/tasks/{id}/status/ -- Toggle / Set Task Status [Assigned User / Admin]

Purpose: Toggles or sets task status [Pending <-> Completed]. Completed tasks record timestamp & exit active reminders.
Headers: Authorization: Bearer <access_token> | Content-Type: application/json

REQUEST BODY [JSON]:

```
{
  "status": "Completed"
}
```

Note: If 'status' is omitted, toggles automatically.

SUCCESS RESPONSE [200 OK]:

```
{ "message": "Task marked as Completed.",
 "task": { "id": 101, "status": "Completed",
  "completed_at": "2026-11-05T12:45:00Z" } }
```

Error Responses: 400 Bad Request: {"error": "Invalid status. Must be 'Pending' or 'Completed'."} | 404 Not Found

DELETE /api/tasks/{id}/ -- Soft Delete Task from Active Manager [Assigned User / Admin]

Purpose: Soft-deletes task [is_deleted=True]. Preserves client contact details and complete communication history.
Headers: Authorization: Bearer <access_token>

SUCCESS RESPONSE [200 OK]:

```
{ "message": "Task deleted successfully." }
```

Error Responses: 404 Not Found: {"error": "Task not found or access denied."}

GET /api/tasks/available-leads/ -- Available Assigned Leads for Task [Role: Authenticated]

Purpose: Returns leads assigned to the user without an active pending task, enabling one-tap creation from mobile drawer.
Headers: Authorization: Bearer <access_token>

SUCCESS RESPONSE [200 OK]:

```
{ "available_leads": [
 { "id": 2, "company_name": "Ss Technologies", "contact_person": "Vikas Patel", "phone": "9742242394",
  "email": "sstech@gmail.com", "city": "Bangalore", "display_name": "Ss Technologies - Vikas Patel" } ] }
```

Mobile Integration: Populates the lead selection bottom-sheet when the representative taps the floating [+] action button.

DATA INTEGRITY & TASK ISOLATION FOR MULTI-ASSIGNED LEADS

* When a lead is assigned to 3 users [Dhruvi, Juhi, Tech Admin], 3 separate records exist in the 'lead_tasks' table.
* Dhruvi can mark her task as Completed without impacting Juhi's pending follow-up.
* Follow-up dates and discussion agendas are isolated per user; each user sees only their own follow-up date.
* If a user is unassigned via DELETE /api/leads/{id}/assignments/{uid}/, their task is soft-deleted [is_deleted=True].
* If the user is re-assigned later, the existing task is seamlessly reactivated with full previous history intact.
* Admins can filter GET /api/tasks/?user_id={id} to inspect any representative's active pipeline.

5. LEAD COMMUNICATION & FOLLOW-UP APIS [CALL, WHATSAPP, EMAIL]

POST /api/tasks/{id}/calls/ -- Record New Interaction & Schedule Follow-up [Assigned User / Admin]

Purpose: Records Phone Call, WhatsApp, or Email notes. Updates task schedule and automatically computes reminder_date.
Headers: Authorization: Bearer <access_token> | Content-Type: application/json

REQUEST BODY [JSON]:

```
{
  "interaction_type": "WhatsApp",
  "interaction_date": "2026-11-05",
  "communication_notes": "Sent product deck",
  "next_followup_date": "2026-11-10",
  "next_followup_notes": "Call for feedback"
}
```

SUCCESS RESPONSE [200 OK]:

```
{ "message": "WhatsApp interaction recorded",
 "task": { "id": 101, "lead_id": 1,
  "next_followup_date": "2026-11-10",
  "reminder_date": "2026-11-09",
  "future_discussion": "Call for feedback" } }
```

Historical Preservation: Never overwrites prior interactions; inserts a new chronological 'TaskCallHistory' entry.

Supported Channels: 'Call', 'WhatsApp', 'Email' [case-insensitive with automatic normalization].

Error Responses: 400 Bad Request: {"error": "Communication notes are required."} | 404 Not Found

GET /api/tasks/{id}/history/ -- Task-Specific Communication History [Assigned User / Admin]

Purpose: Complete chronological interaction log for this representative's task, notes, channels, and follow-up dates.
Headers: Authorization: Bearer <access_token>

SUCCESS RESPONSE [200 OK]:

```
{ "task_id": 101, "lead_id": 1, "company_name": "Apex Healthcare Ltd", "count": 2,
 "history": [
  { "id": 202, "interaction_type": "WhatsApp", "interaction_date": "2026-11-05",
   "communication_notes": "Sent product deck", "next_followup_date": "2026-11-10", "user_name": "Dhruvi Patel" },
  { "id": 201, "interaction_type": "Call", "interaction_date": "2026-11-04",
   "communication_notes": "Initial discussion", "next_followup_date": "2026-11-05", "user_name": "Dhruvi Patel" } ] }
```

Error Responses: 404 Not Found: {"error": "Task not found or access denied."}

Mobile UI Usage: Displays under the 'PREVIOUS INTERACTION HISTORY' collapsible timeline.

GET /api/leads/{id}/interactions/ -- Lead 360-Degree Team Interaction Log [Assigned User / Admin]

Purpose: Aggregates all communication history across ALL assigned team members for full 360 client visibility.
Headers: Authorization: Bearer <access_token>

SUCCESS RESPONSE [200 OK]:

```
{ "lead_id": 1, "company_name": "Apex Healthcare Ltd", "contact_person": "Raj Shah", "total_interactions": 2,
 "interactions": [
  { "id": 202, "task_id": 101, "interaction_type": "WhatsApp", "communication_notes": "Sent product deck",
   "user_name": "Dhruvi Patel", "user_role": "User", "call_date": "2026-11-05" },
  { "id": 203, "task_id": 103, "interaction_type": "Email", "communication_notes": "Sent formal quote with NDA",
   "user_name": "Juhi Vanjara", "user_role": "User", "call_date": "2026-11-05" } ] }
```

Cross-Team Value: Prevents duplicate communications. Rep A can see what Rep B discussed before calling.

Error Responses: 404 Not Found: {"error": "Lead not found or access denied."}

6. REMINDERS & MOBILE PUSH NOTIFICATIONS APIS

GET /api/tasks/notifications/ -- In-App Follow-up Reminders & Due Today [User: Own | Admin: Team]

Purpose: Returns pending tasks where reminder_date <= today [1 day prior to follow-up]. Flags is_due_today & is_overdue.
Headers: Authorization: Bearer <access_token>

SUCCESS RESPONSE [200 OK]:

```
{ "count": 1, "notifications": [
 { "id": "rem-101", "task_id": 101, "lead_id": 1, "company_name": "Apex Healthcare Ltd",
  "contact_person": "Raj Shah", "phone": "9876543210", "email": "raj@apexhealth.com",
  "next_followup_date": "2026-11-06", "reminder_date": "2026-11-05",
  "future_discussion": "Follow up on contract", "is_due_today": false, "is_overdue": false } ] }
```

Mobile UI Banner: Drives the top header bell notification counter and dismissable alert badges.
Exclusion Filter: Tasks marked 'Completed' or deleted are automatically excluded from reminders.

POST /api/notifications/device-token/ -- Register Mobile Push Token [Role: Authenticated]

Purpose: Binds mobile FCM [Android] or APNS [iOS] device token to user account for background push notifications.
Headers: Authorization: Bearer <access_token> | Content-Type: application/json

REQUEST BODY [JSON]:

```
{
  "token": "fcm_e4k8Lm9_device_token_sample",
  "platform": "android"
}
```

SUCCESS RESPONSE [201 CREATED]:

```
{ "message": "Device token registered successfully.",
  "id": 1, "platform": "android",
  "created": true }
```

Platforms Supported: 'android' [Firebase Cloud Messaging], 'ios' [Apple Push Notification Service].
Error Responses: 400 Bad Request: {"error": "Device token is required."}

DELETE /api/notifications/device-token/ -- Unregister Push Token on Logout [Role: Authenticated]

Purpose: Unbinds mobile push token on sign-out to prevent notifications from appearing on shared devices.
Headers: Authorization: Bearer <access_token> | Content-Type: application/json

REQUEST BODY [OPTIONAL]:

```
{
  "token": "fcm_e4k8Lm9_device_token_sample"
}
```

SUCCESS RESPONSE [200 OK]:

```
{ "message": "Device token unregistered successfully.",
  "unregistered_count": 1 }
```

Security Policy: If token is omitted, unregisters all active tokens for this user upon signing out.

24-HOUR FOLLOW-UP REMINDER TIMELINE ALGORITHM

* Formula: reminder_date = next_followup_date - timedelta[days=1]
* Example Scenario: An interaction is logged on 31 Oct with next follow-up on 5 Nov -> reminder_date = 4 Nov.
* Trigger Window: On 4 Nov at 00:00:00 IST, the task enters 'is_reminder_active = True' status.
* In-App Alert: Representative sees top header notification badge with urgent client company and phone number.
* Push Delivery: Cloud server sends background push notification 24 hours prior to meeting.
* Dismissal Condition: When representative marks task Completed or logs a new interaction, reminder is cleared.

7. ROLE-BASED ACCESS CONTROL [RBAC] PERMISSIONS MATRIX

OPERATIONAL CAPABILITY	SUPER ADMIN / TECH ADMIN	ADMIN [COMPANY]	SALES EXECUTIVE
View All Leads in Organization	YES [All]	YES [Company Scope]	NO [Assigned Only]
Assign / Reassign Multiple Users	YES	YES	NO
Create New Leads Manually	YES	YES	View / Auto-Assign
Delete Leads Permanently	YES	YES	NO
Manage Own Task Manager & Status	YES	YES	YES [Independent]
Record Interactions [Call/WA/Email]	YES	YES	YES [Isolated Notes]
Receive Due / Overdue Reminders	YES [Organization]	YES [Company]	YES [Own Tasks Only]

8. MOBILE APPLICATION INTEGRATION & ARCHITECTURE GUIDELINES

1. JWT AUTHENTICATION HEADER

* Header: Authorization: Bearer <access_token>
* Include header in every authorized mobile request.
* On HTTP 401, call /api/auth/token/refresh/.
* If refresh fails, route user back to Login screen.
* Secure tokens in OS Keychain or EncryptedPrefs.

2. MULTI-USER ASSIGNMENT MODEL

* Leads link to users via lead_assignments table.
* Multiple reps see the same lead simultaneously.
* Each rep maintains an independent LeadTask.
* Rep follow-up notes NEVER overwrite other reps.
* Unassigning rep removes lead from ONLY that rep.

3. ACTION LINKS & DEEP LINKING

* Direct Phone: tel:<phone> launches native dialer.
* WhatsApp: wa.me/<num>?text=Hello%20<name>
* Email: mailto:<email>?subject=Follow-up...
* Mobile app can launch communication apps directly.
* Automatically prompts rep to log notes afterward.

4. STATUS CODES & ERROR RESPONSES

* 200 OK / 201 Created: Standard JSON payload.
* 400 Bad Request: Validation error details.
* 401 Unauthorized: Invalid or expired token.
* 403 Forbidden: Insufficient role permissions.
* 409 Conflict: Duplicate phone or email exists.

DOCUMENT STATUS: APPROVED FOR PRODUCTION MOBILE IMPLEMENTATION

BACKEND ENGINE: Django 5.x / Django REST Framework / MySQL Relational Engine

MOBILE PLATFORMS: iOS 14+ / Android 10+ [React Native / Flutter Cross-Platform]

SECURITY & COMPLIANCE: Encrypted Token Storage, Role-Based Access Control [RBAC], Audit Trails

VERIFICATION: 100% verified against Impression CRM backend, automated test suite, and database models.